

Use the data from the workshop to take advantage of reviewers' different perspectives. Share the data and ask reviewers to look for trends. Ask reviewers what worries them. Collect their questions. Even if we know the answer to a question, the fact that someone asked implies there is room to improve communication.

Once we've analyzed the data and reached some conclusions, it's time to decide what to do about it.

Decide on Follow-up Actions

We don't have to address every issue, risk, and open question identified during an evaluation. We won't have time to fix everything. Prioritize the work that must get done and separate it from the issues that are interesting but not essential. Assign someone to decide what to do about each high-priority item.

To close the evaluation workshop, create a summary of the findings and follow-up actions and share the list with all participants. For smaller workshops, a simple email with action items works great. For larger workshops, a brief write-up with links to raw notes ties a nice bow around the evaluation.

The concluding write-up is an excellent way to summarize findings for stakeholders and acts as a visible sign of progress for the architecture. Summaries are an excellent resource both for future architects of this system and architects who want to run evaluation workshops for a different system.

The Gold Standard: Architecture Trade-off Analysis Method (ATAM)

The *Architecture Trade-off Analysis Method* (ATAM) is among the first and certainly the most comprehensively defined and studied architecture evaluation method created to date. The method is described in detail in [ATAM: Method for Architecture Evaluation \[KKC00\]](#) and [Software Architecture in Practice \[BCK12\]](#). The ATAM heavily influenced the basic evaluation workshop outline discussed in [Host an Evaluation Workshop, on page 166](#).

By the book, the ATAM is a multiday, multiweek process that recommends using a trained facilitation team. If you're designing a highly complex system such as missile guidance or autonomous vehicle navigation systems, then the traditional ATAM is highly appropriate. For everyone else, we can usually get away with less process. Use the evaluation workshop structure from this chapter as a starting point.

The ATAM is thorough and well defined. If you've never hosted an architecture evaluation, consider trying an ATAM. It is the gold standard of evaluation methods.

Evaluate Early, Evaluate Often, Evaluate Continuously

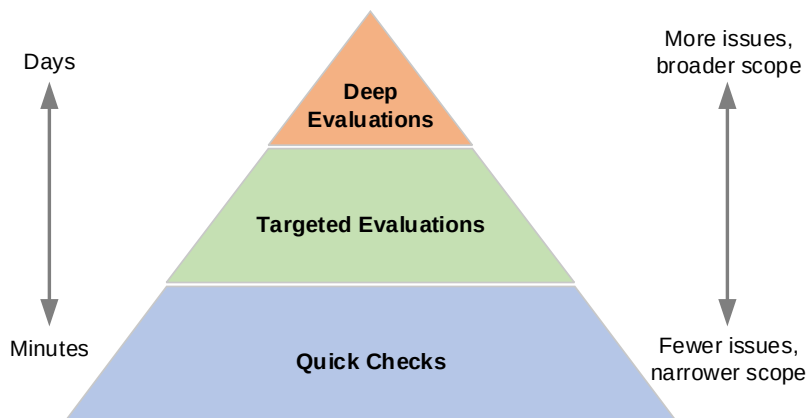
If the first mistake is skipping architecture evaluations, the second mistake is waiting too late to start. The sooner you start testing your designs, the sooner you'll be able to improve them. Better still, make evaluation a regular part of your development routine.

There are dozens of opportunities every day to confirm (or amend) design decisions. Every day we walk through the architecture and tell stories about how it promotes quality attributes. We submit code for peer review. We pair program as a regular part of our everyday workflow.

Balance Cost and Value with the Evaluation Pyramid

The test pyramid is a concept introduced by Mike Cohn in [Succeeding with Agile: Software Development Using Scrum \[Coh09\]](#). The premise is straightforward. Different kinds of tests find different kinds of defects, but some tests are easier to create and maintain than others. The test pyramid proposes that the majority of tests should be fast to run and cheap to maintain unit tests. Since unit tests can't catch everything, we should also create a small number of slow, brittle full-stack integration tests.

The premise behind the *evaluation pyramid* is similar to the test pyramid. The vast majority of architecture evaluations should be fast and cheap *quick checks*. Quick checks won't find every kind of design issue so we'll also want to perform a few thorough but costly *deep evaluations*. To provide for some consistency between quick checks and deep evaluations, we can perform *targeted evaluations*.



On a typical system, we might perform only one or two deep evaluations, dozens of targeted evaluations, and hundreds of quick checks. Deep evaluations will always be a major milestone in the system's life. Targeted evaluations might happen as part of a regular cadence, perhaps as often as every 2 to 4 weeks. Quick checks occur daily—sometimes multiple times each day—and become a seamless part of the development workflow.

Here are some examples of each type of evaluation from the pyramid:

Evaluation Type	How many?	Description	Examples
Deep Evaluation	1–3	Considers the whole system and interplay of several ASRs	Architecture Trade-off Analysis Method
Targeted Evaluation	dozens	Considers a single decision, component, or ASR	Risk storming, Question–Comment–Concern, Architecture Briefing
Quick Check	countless	Considers discrete design decisions as they are made, often used to reinforce understanding or evaluate details	Code review, storytelling, whiteboard jam, sanity check

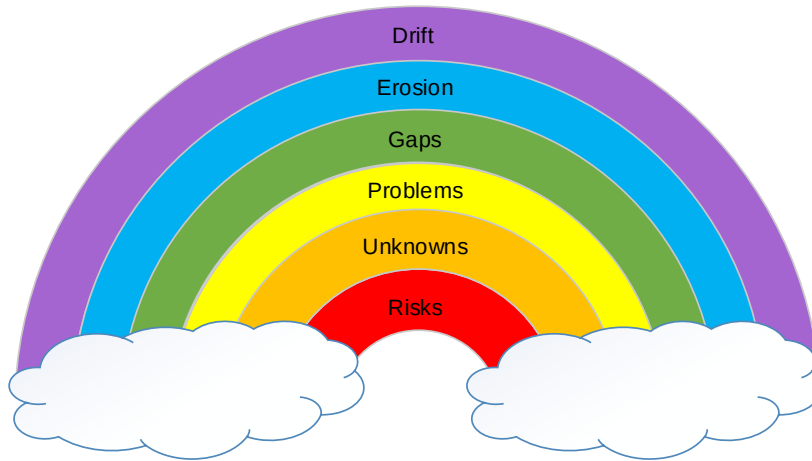
Just because we evaluate the architecture continuously, it does not mean we are doing a good job of it. We also need to look for a variety of issues during our evaluations to ensure we have good coverage.

Look for Different Kinds of Issues

Eat the rainbow is something I tell my son to make sure he eats a variety of fresh fruits and vegetables. Fresh foods are different colors because they have an abundance of different vitamins. Eating different colors ensures he gets all the vitamins and minerals a healthy body needs. Variety is as important for healthy architectures as it is for healthy bodies.

Most of the methods discussed in this chapter and in [Chapter 17, Activities to Evaluate Design Options, on page 285](#) can be adapted to draw out different kinds of issues. An issue, in the general sense, is any topic that requires additional investigation or thought. To ensure we have a healthy architecture, look for issues from across the *architectural issues rainbow* as shown in the [figure on page 173](#).

Architectural Issues Rainbow



Every issue from the rainbow can tell us something different about our architecture.

Risks (Red)

A risk is something bad that *might* happen, but it hasn't happened yet. As you learned in [Let Risk Be Your Guide, on page 32](#), risks have two components: a condition and a consequence. The condition is something currently true about the architecture. The consequence is something bad that might happen as a direct result of the condition.

Risks can be mitigated or accepted.

Unknowns (Orange)

Sometimes we simply don't have enough information to say whether or not the architecture satisfies ASRs. Identify unknowns by looking for open questions about how things work and how specific ASRs will be addressed. Architecture evaluations can turn unknown unknowns into known unknowns. You can deal with the latter. The former can kill your architecture.

Unknowns require further investigation.

Problems (Yellow)

Problems, unlike risks, are bad things that have already come to pass. Problems arise when you make a design decision and it just doesn't work out the way you hoped. Problems can also arise because the world changed around you so that a decision that was a good idea at the time no longer

makes sense. If the architecture already exists in code, we think of problems as technical debt.

Problems can be fixed or accepted.

Gaps in Understanding (Green)

When you zig and your team zags there is a gap in understanding. Gaps in understanding arise when what stakeholders think they know about the architecture doesn't match the current design. In rapidly evolving architectures, gaps can arise quickly and without warning.

Gaps in understanding can be addressed through education.

Architectural Erosion (Blue)

The implemented system almost never turns out the way we imagined it. This gap between the designed architecture and the as-built architecture is called *architectural erosion*, sometimes called *architectural drift* or *architectural rot*. Without vigilance, the architecture drifts from the planned design a little every day until the implemented system bears little resemblance to the plan.

Architectural erosion can be addressed by paying attention to the little details—in code or documentation—on a regular basis.

Contextual Drift (Violet)

Sometimes the world changes without us noticing. Software takes time to build. Over the months, facts that were true can become untrue. New facts come to light. Circumstances change. *Contextual drift* happens any time the business drivers or context driving our decisions changes after we've made a design decision.

Contextual drift can be addressed by occasionally revisiting business goals, architecturally significant requirements, and other things we think we know about our stakeholders and their needs.

A common mistake new software architects make is to look for the same kinds of issues all the time. A simple way to take your architecture evaluations to the next level is to ask questions that haven't been asked before. Look for a variety of issues and you won't be surprised by what you don't know.

Start with Low Ceremony Evaluation Methods

The amount of ceremony in a method refers to how much formality is required to apply the method. High ceremony methods are filled with rituals and can be costly to use. High ceremony methods are easy to repeat and produce consistent results. Low ceremony methods, on the other hand, are informal

and have few rituals. As such, they are faster and cheaper to apply but narrower in scope, and more likely to produce inconsistent results.

If your team is new to architecture evaluations, starting with a high ceremony method such as the ATAM can be off-putting and might scare teammates away from evaluations for life. Instead of diving into high ceremony methods first, ease your team in to evaluations by starting with low ceremony methods. You might choose not to tell your team they are doing an architecture evaluation.

Here's an example of how this could play out. After a whiteboard jam ([introduced on page 255](#)) but before the group disbands, kick off an evaluation. Grab the whiteboard marker and say, *This looks like a good start. Are there any issues you see with our ability to promote <insert quality attribute here>?* Write down what they say on the whiteboard next to the diagrams. Help the team summarize findings and decide on next steps. Boom. Impromptu architecture evaluation completed in 10 minutes or less.

Low ceremony evaluation methods reinforce architectural thinking among the team and build a culture that challenges design decisions. As your team becomes comfortable with low ceremony approaches, strategically introduce targeted evaluations and, eventually, a deep evaluation.

Project Lionheart: The Story So Far...

We're about two months into the project and we've accomplished quite a lot. We populated our backlog, completed an architecture spike, and completed several iterations of development. Continuous integration is running, we finished several value adding stories, and we made our first internal releases. The planned architecture is sketched on whiteboards around the office and several important design decisions are recorded as architecture decision records. You think now seems like a good time to reexamine our planned design based on what we've learned so far.

You schedule a [question-comment-concern workshop on page 298](#) during the next iteration. You start the workshop by reviewing the top five quality attribute scenarios. Next you ask us to draw some views of the architecture related to those quality attributes. Walking us through each scenario, you invite us to add sticky notes to the whiteboard drawings, capturing our questions, comments, and concerns. After 50 minutes we have a long list of issues that needs our attention.

To finish the workshop, you help the team *think* about next steps. We spend the last 10 minutes of the workshop creating an action plan to address the

major issues we identified. Some of the actions include refactoring code that has drifted from our planned design and running an experiment to verify that we can correctly do phased deployments with no downtime.

You schedule another evaluation workshop two iterations as another checkpoint. We discuss design issues daily and everyone decides we should continue encouraging this practice.

Next Up

The architecture is a living part of the team's culture. Every day we choose to live with the architecture we've designed. Living with a poor design can become unbearable. Architecture evaluations help us understand how to make our architecture better.

In the next chapter, we're going to pull together everything you've learned so far and focus on how to lead a team of architectural thinkers to develop awesome software.

Understand

Explore

Evaluate

Make

CHAPTER 13

Empower the Architects on Your Team

It takes a village to develop a modern software system. Technology advances such as containerization, super cheap computers, and on-demand cloud infrastructure put tremendous power and flexibility directly in developers' hands. In response to these new technologies, emerging architecture patterns such as microservices and function-as-a-service assume developers have a greater awareness of how their decisions influence quality attributes and other system properties.

On modern software systems, there is little difference between a *developer* and an *architect*. This isn't to say that modern software development teams don't have technical leaders. They do, though today's software architects don't always self-identify as architects. Modern software development teams need a different kind of leader than the traditional, top-down architect.

Modern software architects design with their team, not for their team. Today's architects are equal parts coach, mentor, and technical guru. We started this book by discussing essential architecture and design principles. Throughout Part II you learned how to put those principles into practice. In this chapter, you'll learn how to grow and empower your team as you design awesome software architectures together.

Promote Architectural Thinking

Teams who embrace the idea of *software architect* as a way of thinking instead of as a role on the team produce better software. When the majority of the team is an architect, the team can explore more design decisions faster. Software quality increases since more eyes can critically evaluate design decisions. Documentation is leaner and conveys more knowledge with less effort.

When everyone on the team can design architecture, everyone feels a greater sense of shared ownership over the design. The software becomes *our system*, not *the system*. Change becomes easier to manage since everyone understands the intent behind the design and feels a sense of responsibility for maintaining design integrity. Development velocity increases thanks to less rework, improved quality, and more efficient communication.

With a greater sense of ownership comes increased happiness, which in turn amplifies engagement in the whole software development process. When we design software together, we create force multipliers that accelerate our ability to ship amazing software. The benefits are amazing, but it's important to remember not everyone is ready to accept more design responsibilities.

It is our job to nurture our team's design skills while at the same time designing an architecture with them. We must empower our team while at the same time ensuring we design an appropriate architecture that lets us ship value-adding software for our stakeholders.

We accomplish this by providing just enough guidance to keep the team on track without requiring that we oversee every design decision. We build skills. We improve trust. We make it possible to recover from mistakes. We try to stay a few steps ahead of our team so we can shepherd them away from traps and pitfalls.

Programmers Make Architectural Decisions Every Day

In the early 2000s I read a software horror story about a developer who managed to take down all phone switches on the Eastern seaboard with a single line of code. *That'll never happen to us!* I told a coworker. We laughed. Fifteen years later the joke is on me.

These days I don't have to read case studies to find software horror stories. I only need to talk to fellow developers. I have met a developer who accidentally spent over \$20,000 on Amazon Web Services for a university project. Another who spent a weekend recovering terabytes of data after a script did not behave quite as expected. Me? I once took out a cluster of servers used by over 100 developers with one line of code.

Today's programmers make design decisions about the architecture every single day. When one line of code can tank a required quality attribute, then you are a software architect whether you identify as one or not.

Facilitate Decision Making and Foster Skills Growth

For the team to truly own the architecture, the software architect must support them fully. Instead of acting as the sole design authority, we'll infuse our teams with the knowledge and skills they need to make design decisions for themselves. When things are going well, architects look more like coaches or mentors than authoritative leaders who make all the design decisions. Above all else, when possible, we let the team design the architecture instead of making the decisions ourselves.

Here are some examples of how architects can make this happen:

OK Software Architects	Great Software Architects
Select patterns and technology without input	Collaboratively select patterns and technologies with input from the team
Write detailed documents, release only once, fully complete	Create document templates for the team to use, build and review documents with the team incrementally
Make or approve all design decisions	Teach the team how to decide, provide design guidance, delegate decision making, provide reviews and feedback
Dictate who builds specific elements	Help the group self-organize and choose work
Avoid changes to the architecture	Embrace the inevitability of change and make the architecture easy to change
Mandate technology decisions	Build consensus for technology decisions

It's never easy to replace a great software architect, but great software architects can leave their team when the time is right. Software architects can safely move to a new team, not because they finished all the hard design work, but because the team has learned what it takes to be great software architects themselves. The only way this can happen is with practice.

Create Opportunities for Safe Practice

We want to give our team more design authority, but this is only possible if they're prepared to handle it. Teammates can't gain experience unless they practice. We're on a tight deadline, and taking time away from development for training is asking a lot. What do you do? Find safe ways to practice design.

Pair Design

Pairing is one of the simplest and safest ways to practice design. Start by working one-on-one with teammates while you're doing design work. If you need to think through a model, invite a teammate to join you for a whiteboard jam, [introduced on page 255](#). If you have a meeting to talk to stakeholders about quality attributes, take a teammate with you. Ask a teammate to review a document filled with decisions before you share it with the whole team.

Create Scaffolding

In education theory, *instructional scaffolding* refers to support structures given to individual students to promote and reinforce learning. In school, you likely experienced many scaffolding techniques such as detailed feedback on a test, handouts, rubrics, and homework templates. We can use similar techniques when teaching our team about architecture. Here are a few examples:

- Build templates to support commonly delegated design work.
- Provide constructive criticism during peer reviews. See [Activity 31, Code Review, on page 289](#).
- Create a code skeleton for new components. The code skeleton should sketch the module patterns planned but still require work to put meat on the bones. Bonus: Pair with someone to create the skeleton.
- Describe expectations for a particular artifact and share an example of what *better* and *worse* versions of that artifact look like.
- Create checklists for different design mindsets and tasks to help teammates internalize architectural thinking.

Introduce Architectural Guide Rails

An *architectural guide rail* restricts design options to ensure detailed design stays within the bounds of the desired architecture. Guide rails are a form of constraint we choose for ourselves. We can use guide rails to make design simpler (see [Limit Design Options with Constraints, on page 49](#)), but we can also use guide rails to create opportunities for safe practice. Imposing guide rails decreases the chances that we'll mess up the architecture.

Guide rails come in many forms and varying strengths. A *design policy*, instructions that describe something to do or avoid doing, is a simple guide rail but difficult to enforce. We can temporarily impose design policies when

we delegate design work or put policies permanently in place to reduce risk, promote quality attributes, or overcome team weaknesses.

The strictest architectural guide rails are built into the code and make it impossible to do the wrong thing in the architecture. We discussed several approaches for creating guide rails in [Build Models into the Code, on page 107](#). One example of a guide rail is to require the use of a specific library. Imposing this constraint might make development easier and help you avoid simple mistakes.

Host Information Sessions

If your team is keen on software architecture, consider hosting information sessions so you can dive deep into specific topics. Always be prepared to teach relevant information just-in-time before the knowledge is required to be applied. Dozens of micro-lessons are just as good as a single long training session.

As the team's skills increase, they'll start to share more feedback about the architecture's design. Be gracious with feedback and encourage it. When the team is willing to tell you how to improve the architecture, then you'll know you're doing something right and can think about delegating more design authority.

Delegate Design Authority

As we include more of the team in the design process, we must decide how much design authority to keep and how much to delegate. Our goal is to give away as much design authority as possible without putting essential quality attributes at risk or otherwise endangering the architecture. In [Management 3.0: Leading Agile Developers, Developing Agile Leaders \[App11\]](#), Jurgen Appelo describes seven levels of authority. We can use these levels to help us decide how much design authority to keep and what we can leave to the team.

Level 1: Tell You make the design decision and tell the team what will happen, usually by producing an artifact.

Level 2: Sell You make the design decision and show the team why it is the right call.

Level 3: Consult You ask the team for input before making the design decision. Ultimately the decision is still yours.

Level 4: Agree You collaborate with your team and reach consensus about the design decision as a group. Everyone has an equal voice.

Level 5: Advise You influence the team by sharing your opinions and insights but let someone else make the design decision.

Level 6: Inquire You let the team make the design decision and ask them to show why their decision is the right one.

Level 7: Delegate You leave the design decision to someone else. In this capacity, you might help gather information as a facilitator but someone else is responsible for the decision.

The level of authority you use will vary from decision to decision and team to team. Delegate the right level of design authority for the situation and you'll increase the team's confidence, happiness, and agility. Delegate too little design authority and you may undermine trust and make some people feel micromanaged. Delegate too much design authority and you'll end up with an anxious, unhappy team and a poor design.

In the best case, when you delegate too much design authority to a team that is not ready to handle it you'll have to try again at a lower authority level. When the team fails under these circumstances, it undermines trust and creates waste through rework, assuming you catch the decision early enough. Although that situation is not ideal, you can still recover. In the worst case, a bad design decision might go unnoticed until it's too late to recover easily.

Choosing the appropriate level of design authority is not an exact science. It takes some trial and error to figure out how your team likes to operate. The easiest way to check that you're choosing the best level of authority is to talk it over with your team and decide together.

When to Keep Design Authority

When the risks of failure are high, it's better to be conservative with how much authority you delegate. Stick to the first three delegation levels when the team is inexperienced. This approach is proven to work and improves your chances of producing a useful design. Unfortunately, the first three delegation levels don't come with as many bonus multipliers in quality, happiness, speed, and agility.

Simultaneously designing an architecture and skilling up a team is one of the most challenging times to be an architect. You're under pressure to deliver and unblock development. It'd be so much easier just to design it yourself! In the short term this may be true, but then your team will not grow and you will never be able to delegate design decisions. If you are the only

architect on your team, then your skills, knowledge, and time will constraint the types of systems your team designs.

If you are in doubt, keep design authority until an appropriate opportunity presents itself.



Patrick says:

Architect as a Technical Leader

by Patrick Kua, principal technical consultant at ThoughtWorks

An architect's role is tough. Amongst their many responsibilities, an architect aims to reduce technical risk, plan for future change, and ensure that systems meet their quality attributes. However, for an architect to be truly successful, they also need to act like a technical leader.

Effective architects cannot always rely on their authority to make decisions. In today's ever-changing technology landscape, it's impossible for an architect to know all the details about the latest tools and technologies and how to apply them well. Instead, the effective architect must draw upon the wider experience of their development team and organisation. As a technical leader, the architect builds a shared technical vision for the team and focuses on amplifying the effectiveness and growing the skillsets of team members.

If architects are measured by the quality of decisions, they should also be interested in helping developers make better decisions. After all, each line of code represents a choice and, ultimately, each choice is a decision that has been made. The architect can improve developers' decision making by articulating operational or environmental constraints and agreeing with the team on architectural principles to gently guide future decisions.

Each of these activities—building a technical vision, describing constraints, and applying architectural principles—requires completely different nontechnical skills. These skills, often classified as “soft skills,” are often the hardest to build. To be successful in this role, architects should draw upon deep communication skills such as explaining technical ideas in non-technical terms, using diagrams and models to build a common understanding and telling stories to motivate, excite, and challenge team members. Another essential leadership skill worth developing is the ability to listen. Not only will good listening skills improve the knowledge of the architect, but they will also grow the commitment of the team and organisation to the final technical vision.

Those architects who solely focus on deep technical expertise and owning all technical decisions are destined to build and live in their own ivory tower. You can avoid this situation by developing technical leadership skills and use them to build stronger bridges with the team and the rest of the organisation.

When to Give Away Design Authority

If your team already has some experience, then look for opportunities to consult, decide together, or only share advice before letting others decide. Delegating design authority is of particular importance for decisions that strongly influence the team's day-to-day-work or for design decisions in which there is passionate interest.

As the team learns more about architecture and as you gain confidence leading the team, incorporate collaborative workshops into your practice. Collaborative group sessions work well when you're able to agree, advise, inquire, or fully delegate decision making. Your role as a participant will decrease as you give more authority to the team. Instead, you support your team as a knowledgeable facilitator.

Many activities in Part III describe ways to include multiple stakeholders in the decision-making process. Here are a few examples:

- Tell stories with your team and encourage them to tell stories about the architecture too. Check out [Anthropomorphize the Architecture on page 226](#) and [System Metaphor on page 281](#).
- Encourage active participation with collaborative workshops such as [Design Studio on page 113](#), [Question-Comment-Concern on page 298](#), [Risk Storming on page 301](#), and [Scenario Walkthroughs on page 307](#).
- Check that the team is engaged and understand what's happening with an [Architecture Briefing on page 286](#) or [Sanity Check on page 304](#).
- Delegate artifact creation when good examples are available. A few artifacts that are easy to delegate and review include [Architecture Decision Records on page 260](#), [Architecture Haiku on page 263](#), and [Inception Deck on page 269](#).



Get Your Hands Dirty: Delegation Poker

In [Managing for Happiness: Games, Tools, and Practices to Motivate Any Team \[App16\]](#), Jurgen Appelo introduced *Delegation Poker*, a game you can play with your team to practice choosing a delegation levels. Print or buy *Delegation Poker* cards from the Management 3.0 website¹ and play this game with your team. The game's rules are available on the website.

1. <https://management30.com/product/delegation-poker/>

Here are some things to think about:

- Before playing, each player should write down a few brief case studies related to areas you want to agree on a delegation policy. Try to have at least one case study from each teammate.
- Use the chapters in Part II to guide topic areas. Does the team feel confident in their skills for some topics?
- Are there decisions the team does not feel comfortable owning? Why?
- What areas will benefit most from giving the team more design authority? How can you help your team prepare for the additional responsibility?

Design Architecture Together

You learned in [Build Amazing Software, on page 12](#) how software architecture improves our ability to ship awesome software. The architecture, not the architect, provides these benefits. It's the architect's job to guide the team to design an architecture so we can get these benefits. In [What Software Architects Do, on page 3](#) you learned how software architects go about this task, and we've expanded on these ideas throughout the book.

Let's revisit the architect's key responsibilities from the perspective of our new knowledge:

Define the problem from an engineering perspective. Architects are responsible for defining the architecturally significant requirements, especially quality attributes. We prefer to use human-centered design methods to gather these requirements so we don't lose touch with stakeholders' true needs.

Partition the system and assign responsibilities to elements and teams. Architects guide the team to identify patterns that will promote desired quality attributes. We prefer to minimally design the architecture to ensure key quality attributes are achieved, leaving all other decisions to downstream designers.

Keep an eye on the bigger picture and ensure design consistency of the whole. Architects monitor the design as it emerges and shepherd the team as they implement the architecture. We strive to capture design decisions as accurate models using the lightest-weight documentation methods that work for the team and stakeholders. We use these models to reason about the system, evaluate our decisions, and identify risks in our ability to achieve business goals.

Decide trade-offs among quality attributes. Architects help the team reconcile trade-offs as design decisions are made and as the architecture evolves over time. We use risk to determine how much design work is required and where to focus the team's attention.

Manage technical debt. Architects deal with the reality of shipping software by identifying technical debt and devising strategies for paying it back. We recognize that technical debt is an unavoidable consequence of success and work to manage debt strategically over the life of the software system.

Grow the team's architecture skills. Architects empower their teams to take ownership of the system by ensuring the team has the knowledge and skills required to understand, explore, make, and evaluate an architecture. We prefer to design the architecture with our team instead of designing the architecture for our team.

For many teams programming is the easy part. Understanding what the problem is and deciding how the broader system comes together to solve that problem can be tough. The better everyone understands the architecture, the more prepared your team will be to tackle the challenges of software development. Designing the system together creates this deeper understanding.

Project Lionheart: The Epic Conclusion

Mayor van Damme is pleased. We completed the project only a few weeks behind our original schedule despite the constant stream of changes that occurred nearly from the start. We satisfied all the high-priority quality attributes, though there were a few hiccups during load testing before the official public release. Other than that everything went well.

The team strongly feels that the foundation provided by our architecture made it all possible. Our decision to do less architecture design up front worked out fine. The team had prior experience with most of the frameworks and technologies we used. In the beginning, we focused on the areas of greatest risk and worked to build consensus around the architecture. We had a few lucky breaks, such as stumbling upon a major problem with two web services, but we found the problems early enough that we had time to rewrite them from scratch. The final iterations of the project were stressful but not unbearably so.

Our last official work on Project Lionheart is to create maintenance documentation for the city's IT department. The team is writing a *moth ball* document, a user's guide, and cleaned-up versions of relevant architecture views. The original architecture driver's specification maybe only 50 percent accurate at

this point. We never created a formal architecture description. We'll retain these documents for historical purposes and create a new *teachability viewpoint* to walk new developers through the architecture as it exists in the code.

The mayor's office estimates the application we built will save the city nearly \$1 million in the first year. It feels good to see how our design decisions help the city meet its goals in the real world.

Next Up

Architects are leaders. This fact does not mean architects choose every element and relation in the architecture by themselves. Software architects strengthen their influence over the design by enabling others. Grow your team's architecture skills by collaborating with them and by creating safe opportunities for practices. This growth is just as important as making good design decisions.

The principles and practices you learned in Parts I and II give you most important things you need to know about software architecture design. Master this information and you will be an amazing software architect. What we covered here is far from the whole story. Many excellent resources are available that go deeper into documentation, viewpoints, patterns, evaluations, and specific technologies. As you grow as an architect, never stop going deeper, learning more.

Your next step? Use what you learned to build amazing software! To help get you started on your journey as an architect, Part III includes a collection of practical design methods organized around the four design thinking mindsets. I call it the *silver toolbox* as an homage to Fred Brooks's seminal [No Silver Bullet \[Bro86\]](#) essay.

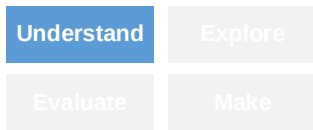
No single software engineering practice provides an order of magnitude improvement in productivity, reliability, or simplicity. Although there are no silver bullets, we all have a *silver toolbox*, a collection of software engineering methods that, when used together, make vast improvements possible. I hope you have already found a few useful tools in this book that you can add to your silver toolbox.

The state of software design practice ten years ago looked very different than it does today. Ten years from now the way we design software systems will be different still. You're now a part of the community who will shape the future. Don't worry—it'll be fun. And we'll build some awesome software along the way.

Part III

The Architect's Toolbox

Every architect has a collection of design methods he or she uses to get stakeholders talking and teammates thinking. Use the methods in Part III to start your own architect's toolbox.



CHAPTER 14

Activities to Understand the Problem

In the understand mindset we actively seek information from stakeholders and work to define (or redefine) the problem. Understanding is more than just specifying requirements. We also need to figure out who our stakeholders are, identify business goals for the system, and ensure requirements specified with an eye toward the architecture.

As you'll recall from [Chapter 5, *Dig for Architecturally*, on page 49](#), there are four kinds of architecturally significant requirements. All of these requirements will influence the architecture, but quality attributes are the most influential and a key concern for architects.

Constraints Unchangeable design decisions, usually given but sometimes chosen

Quality Attributes Externally visible properties that characterize how the system operates in a specific context

Influential Functional Requirements Features and functions that require special attention in the architecture

Other Influencers Time, knowledge, experience, skills, office politics, your own geeky biases, and all the other stuff that sways your decision making

The activities in this chapter help teams empathize with stakeholders and dig for architecturally significant requirements. Use them when you need to get a better grasp on the real problem.

Activity 1

Choose One Thing

Discuss priorities with stakeholders by presenting them with an extreme choice: if you only get one thing, what will it be? This activity can help stakeholders make decisions when facing difficult trade-offs.

Benefits

- Clearly communicates, *this is more important than that*.
- Starts a conversation about why a certain choice was made and what would need to change for stakeholders to change their mind.
- Makes it obvious when stakeholders disagree.

Participants

All stakeholders

Preparation and Materials

- List of alternatives, such as quality attributes, or other difficult trade-offs, such as cost, schedule, and features.

Steps

1. Explain the rules of the game. Stakeholders may choose only one of the presented options. Remind participants that this does not mean you will do only one thing, but instead the point is to have tough conversations early to avoid trouble down the road.
2. Present a set of options to stakeholders. Discuss what each option means and make sure everyone understands the options.
3. Force stakeholders to pick one option. All stakeholders should agree that this is the most important thing. In this activity we want consensus.

4. Briefly discuss why the option was selected. This discussion is often more important than the option picked.
5. Pick another set of architecturally significant requirements that are in tension with one another and play again.

Guidelines and Hints

- Play this game before things get too difficult. This conversation is easier when it's hypothetical than when it's real.
- Quality attributes that are in tension with one another should be pitted against one another.
- Use this activity to prioritize influential functional requirements.
- This technique works well as an informal conversation to help get better understanding of stakeholders' true need.

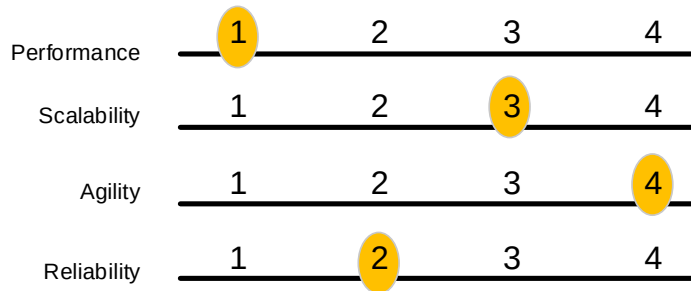
Example

Here are a few scenarios one team posed to some stakeholders and how things played out when stakeholders were asked to choose one thing:

Matchup	Stakeholder's Choice
Faster performance or greater accuracy	Faster performance, assuming accuracy met a required minimum threshold.
Cost vs. time-to-market	Time-to-market; stakeholders were willing to accept greater technical debt to get required features by a specific date.
Usability vs. security	Security; this was the number one quality attribute and surprisingly beat several other important quality attributes.
Availability vs. cost	Availability; achieving high availability on this particular system required potentially expensive redundancy for which stakeholders would willingly pay (up to a point) if required.

Alternatives

Instead of pitting one alternative against another, use *trade-off sliders* to let stakeholders make multiple comparisons. To use this activity, identify 3–5 related alternatives. Each item can receive a number 1 – N, where N is the number of items in the list. No two items can have the same number. This activity is often presented visually using sliders.



Activity 2

Empathy Map

Brainstorm and record a particular stakeholder's responsibilities, thoughts, and feelings to help the team develop a greater sense of empathy with stakeholders' goals.

Benefits

- Discover your audience's needs before developing an architecture description
- Help decide what information to include or exclude
- Create a rubric for evaluating the effectiveness of an architecture description

Activity Timing

10–30 minutes

Participants

Software architect, development team

Small groups of 3–5 or as a solo exercise

Preparation and Materials

- Before the activity starts, choose which stakeholders, systems, or users will be the focus of the activity.
- Flipchart paper or a whiteboard, markers, and sticky notes
- This activity can be adapted for remote participants with screen-sharing or remote collaboration software.

Steps

1. Draw a grid on a whiteboard or piece of paper. Label each quadrant—do, make, say, and think.
2. Pick a specific stakeholder and write his or her name in the middle.

3. Brainstorm tasks this person does, artifacts this person makes, things this person says, and feelings this person may have.
4. Write each idea on a sticky note and place it in the corresponding quadrant.
5. Review the empathy map and highlight insights.

Guidelines and Hints

- Be specific. Pick a person to empathize with, not a general role.
- Validate the findings from your empathy map with stakeholders.
- Mention quality attributes, risks, or other concerns this person may find relevant.
- Adapt this method for understanding application end users, external systems (for interface design), or for use with proxy stakeholders to understand quality attributes.
- Use software such as Mural¹ when participants are distributed.

Example

There is an example empathy map for a developer stakeholder [persona on page 197](#).

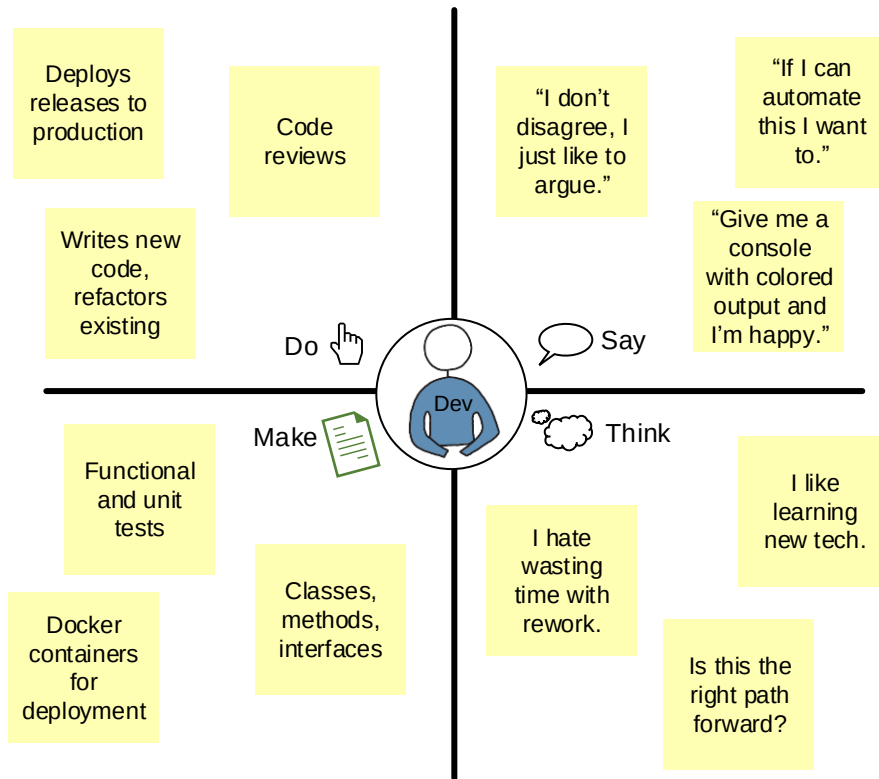
Alternatives

The quadrants of an empathy map can be changed. Another common schema is hear, see, do (or say), and think (or feel).

Empathy maps are also useful for quality attribute analysis.² This approach is especially useful when your stakeholders are unable to participate in other workshops, such as the [Activity 7, Mini-Quality Attribute Workshop, on page 210](#). Instead of focusing on what the stakeholders do, say, or think, we focus on how stakeholders react to specific quality attributes. Obviously, it's better to ask stakeholders directly. When they are not available, this is a good substitute.

1. <https://mural.co/>

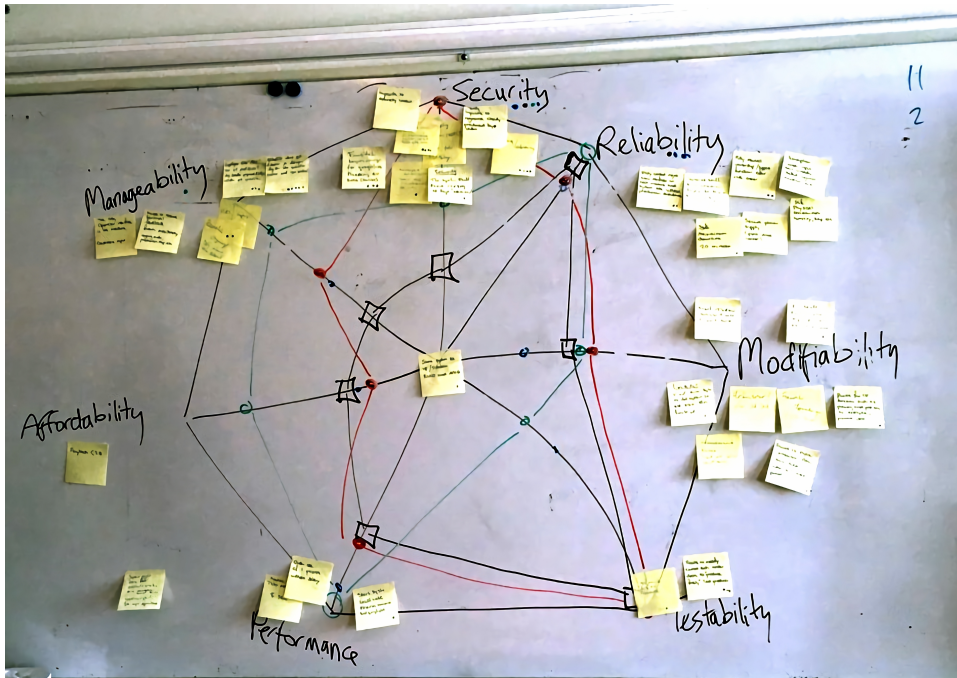
2. Thijmen de Gooijer. *Quality Requirements on a Shoestring*. SATURN 2015.
<http://resources.sei.cmu.edu/library/asset-view.cfm?assetID=436426>



To use this variant, pick a stakeholder and brainstorm at least two quality attribute scenarios or general concerns for each relevant quality attribute. Use dot voting³ to estimate how this stakeholder might rate the quality attributes. Ideally, you should validate the outcomes of this exercise, but this is not always practical. These insights can be used during other workshops to help ensure a stakeholder's perspective is represented even when that stakeholder is not present.

3. <http://dotmocracy.org/dot-voting>

In the following example, from a workshop facilitated by Thijmen de Gooijer, sticky notes represent raw quality attribute scenarios. The lines in the center of the chart show how different absentee stakeholders might have felt about different quality attributes shown on a [quality attribute web on page 207](#). The quality attribute web helps us visualize the importance of different quality attributes during structured brainstorming activities. During the workshop, different participants were asked to play the role of an absentee stakeholder by using the empathy map as a guide.



Activity 3

Goal-Question-Metric (GQM) Workshop

An approach for identifying metrics and response measures so that we can connect data with business goals. The *Goal-Question-Metric approach* (GQM) was introduced by Victory Basili, Gianluigi Caldiera, and H. Dieter Rombach in *The Goal Question Metric (GQM) Approach [BCR94]*. The goal of GQM is to identify measures we can use to determine whether a goal has is satisfied.

There are three parts to the GQM approach. The goal defines conceptual requirement that must be met. Goals can describe quality attribute scenarios, general software quality, business goals, or other topics. Questions illustrate the means by which we can characterize one or more goals. Metrics defines the measures needed to answer one or more questions.

Benefits

- Emphasizes using stakeholder goals as the basis for measures
- Shows clear lineage from data to stakeholder goals by way of the questions that must be answered to decide whether a goal is satisfied
- Flexible approach that can help teams think about metrics in a variety of situations

Activity Timing

15–90 minutes

Participants

This activity can be done solo or in a small group of 2–5 people. Any mix of stakeholders will work.

Preparation and Materials

- Whiteboard or flipchart papers, markers
- Goals to be explored can be optionally identified before the workshop

Steps

1. Write a goal at the far left of the whiteboard.
2. Prompt participants to provide questions. *What questions would you need to answer to know if we've met this goal?* Write each question to the right of the goal. Draw lines from the goal to the questions to create a tree.
3. Explore each question to identify metrics needed to answer the question. Write each metric to the right of the questions. Draw lines from each question to the metrics required to answer it.
4. Repeat the exercise for any related goals that might reasonably use the same questions or metrics. The end result should be a tree that connects metrics to questions and questions to goals.
5. Identify data required to compute each metric. Write the data needs to the right of the metrics. Draw lines from each metric to the data needed to compute the metric.
6. For each piece of data identified, determine where you can get the data. Write down each data source for each bit of data. Describe the cost of gathering data from each of the data sources.
7. The last phase of the workshop is to prioritize data and metrics. Clearly identify *must have* metrics. Look for data sources that can provide data needed to compute multiple metrics or metrics that can answer multiple questions.
8. Record the results of the workshop by taking pictures and writing down the discussed goals, questions, metrics, data, and data sources.

Guidelines and Hints

- Be sure to have plenty of space for drawing the GQM tree.
- Look for opportunities for reuse. Metrics can be used to answer more than one question. Likewise, the same data might help compute multiple metrics.
- Data sources and data are likely to influence the architecture, but also look for opportunities to gather data outside of the architecture.
- Record results in a table or spreadsheet to validate with stakeholders later. The identified metrics are revisited throughout the life of the software system.

Example

In this example, the goal is written on the far left, questions are captured in the middle column, and metrics are written in the column on the far right. Data used to compute the metrics was omitted from this particular GQM sketch.

